

ABC450 D - Minimize Range 题解

题意概括

给定 N 个正整数 A_i 和一个正整数 K 。

我们可以任意次选择一个位置，把对应的 A_i 加上 K 。也就是说，每个数都只能不断增大，但它对 K 取模的结果永远不变。

要求最小化最终序列中的 $\max(A) - \min(A)$ 。

解题思路

先把每个数写成

- $A_i = q_i \cdot K + b_i$
- 其中 $0 \leq b_i < K$

因为操作只会加上整倍数的 K ，所以每个数最终的余数 b_i 是固定的。真正能调整的，只有它落在“第几层”。

这提示我们不要盯着原数值，而要盯着所有余数 b_i 在模 K 的圆环上是怎么分布的。

把所有余数从小到大排序为 $b_1 \leq b_2 \leq \dots \leq b_N$ 。再看这些相邻余数之间的环形间隔：

- $b_{i+1} - b_i$ ($1 \leq i < N$)
- $b_1 + K - b_N$

设其中最大的间隔是 max_gap ，那么答案就是：

- $K - \text{max_gap}$

为什么？

因为我们可以把圆环在这段最大空隙处剪开。剪开以后，所有余数会落成一段连续的线段，这段线段的长度正好是 $K - \text{max_gap}$ 。

接下来只要把所有数统一抬到足够高的层数：

- 在线段左边的余数保持不变；
- 在线段右边绕回去的那一部分余数视为“加了一个 K ”；

这样每个数都能通过若干次 $+K$ 落到这条长度为 $K - \text{max_gap}$ 的线段里。

所以问题本质上就是：

- 在模 K 的圆环上，用一段连续弧把所有余数盖住；
- 最短弧长等于“整圈长度减去最大空隙”。

正确性说明

下面分别说明这个做法的上界和下界。

1. 这个答案一定可以构造出来

设最大的环形空隙出现在某个切口处，把圆环从这里剪开。

剪开后，每个余数都会被映射到一条长度为 $K - \text{max_gap}$ 的线段上。设映射后的值为 c_i ，那么：

- 对每个 i ，都有 $c_i \equiv A_i \pmod{K}$
- 并且所有 c_i 都落在同一段长度为 $K - \text{max_gap}$ 的区间内

再设 $M = \max q_i$ 。对于每个数，我们直接取最终值

- $X_i = M \cdot K + c_i$

如果某个 c_i 是原余数 b_i ，那么 $X_i \geq q_i \cdot K + b_i = A_i$ 。

如果某个 c_i 是 $b_i + K$ ，那就更显然有 $X_i \geq A_i$ 。

因此每个 X_i 都能通过若干次加 K 从 A_i 变出来，而且所有 X_i 的最大值与最小值之差正好不超过 $K - \text{max_gap}$ 。

所以答案至多是 $K - \text{max_gap}$ 。

2. 任何方案都不可能更优

考虑任意一种合法方案，设最终所有数都落在一个长度为 W 的区间内。

由于我们已经给出了一个不超过 $K - 1$ 的构造，所以最优解一定小于 K 。这时把最终值对 K 取模，可以把它们看成圆环上的一些点。

如果这些点全部落在一段长度为 W 的区间里，那么圆环上剩下的那段“没被覆盖”的弧长至少是 $K - W$ 。

也就是说，在排好序的余数之间，至少存在一个环形空隙，长度不小于 $K - W$ 。于是

- $\text{max_gap} \geq K - W$

移项得到

- $W \geq K - \text{max_gap}$

所以任何方案的范围都不可能小于 $K - \text{max_gap}$ 。

结合上面的可构造性，答案恰好就是 $K - \text{max_gap}$ 。

复杂度

- 时间复杂度: $O(N \log N)$ ，主要是排序
- 空间复杂度: $O(N)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

const int maxn = 200000;

long long a[maxn + 5];
long long b[maxn + 5];

int main(){
    int n;
    long long k;
    cin >> n >> k;

    // 读入并记录每个数对 k 的余数
    for(int i=1; i<=n; i++){
        cin >> a[i];
        b[i] = a[i] % k;
    }

    // 把余数排好序，方便统计环上的最大空隙
    sort(b + 1, b + n + 1);

    long long max_gap = b[1] + k - b[n];
    for(int i=1; i<n; i++){
        max_gap = max(max_gap, b[i + 1] - b[i]);
    }

    // 最短覆盖弧长 = 整圈长度 - 最大空隙
    cout << k - max_gap << '\n';
}
```

```
return 0;
```

```
}
```

